

보안관제 프로젝트 상세설계서(LLD) v1.0

보안관제 프로젝트 상세설계서(LLD) v1.0

Project: SOC Detection & Monitoring Lab

Document: Low-Level Design

Version: v1.0

Baseline Date: 2026-08-24

Status: IMPLEMENTATION READY CANDIDATE

1. 문서 목적

본 문서는 선행 산출물인

- 요구사항 정의서 v1.0
- 시스템 아키텍처 및 기본설계서(HLD) v1.0

에서 확정된 Architecture를 실제 구축 가능한 값으로 구체화한다.

본 LLD의 핵심 목표는 다음과 같다.

구현 담당자가 추가적인 기술선택이나 구조설계를 수행하지 않고 본 문서의 VM명, Network, IP, Interface Role, Rule Path, Log Path, Port, Volume 및 Test 기준을 따라 구축할 수 있도록 하는 것

2. HLD Baseline

다음 Architecture는 변경하지 않는다.

```
Windows Host
  ↓
Hyper-V
  ↓
Attacker / Gateway / Victim
  ↓
Hyper-V Port Mirroring
  ↓
Security Sensor VM
  ↓
Suricata 8.x
  ↓
eve.json
  ↓
Wazuh Agent
  ↓
Wazuh Manager
  ↓
Wazuh Indexer
  ↓
Wazuh Dashboard
  ↓
SOC Analyst
  ↓
Investigation
```

↓
Detection Tuning
↓
Evidence

3. Technology Version Baseline

2026-08-24 기준으로 다음 Version을 고정한다.

Technology	LLD Version
Suricata	8.0.6
Snort	3.12.2.0
Snort libDAQ	3.0.27
Wazuh	4.14.7
Docker Desktop	4.87.0
MITRE ATT&CK	19.2
Kali Linux	2026.2
Sensor/Gateway/Victim	Ubuntu Server 22.04.5 LTS

Suricata 8.0.6 공식 문서는 AF_PACKET, Passive IDS 및 Offline PCAP을 Tier 1 지원 기능으로 분류한다.

Snort 공식 사이트는 현재 Snort 3.12.2.0과 libDAQ 3.0.27을 제공한다.

Wazuh Docker 공식 문서는 현재 **v4.14.7** tag를 기준으로 single-node deployment를 제공한다.

Docker Desktop 4.87.0은 2026-08-17 release이며 WSL 2 backend는 WSL 2.1.5 이상을 요구한다.

MITRE ATT&CK 현재 Content Version은 19.2이다.

4. Naming Convention

모든 Project Resource는 다음 Prefix를 사용한다.

soc -

4.1 VM Naming

soc-attacker
soc-gateway
soc-victim
soc-sensor

4.2 Hyper-V vSwitch

```
soc-vsw-mgmt
soc-vsw-attack
soc-vsw-victim
```

4.3 Hyper-V NIC

Attacker

```
nic-attack
```

Gateway

```
nic-mgmt
nic-attack
nic-victim
```

Victim

```
nic-victim
```

Sensor

```
nic-mgmt
nic-monitor
```

4.4 Wazuh Container

```
soc-wazuh-manager
soc-wazuh-indexer
soc-wazuh-dashboard
```

4.5 Docker Network

```
soc-wazuh-net
```

4.6 Evidence

```
EV-NET-001
EV-MIRROR-001
```

EV-SURI-001
EV-SNORT-001
EV-WAZUH-001
EV-ANALYSIS-001
EV-TUNE-001

4.7 Test Case

TC-HOST-xxx
TC-NET-xxx
TC-MIRROR-xxx
TC-SURI-xxx
TC-PCAP-xxx
TC-SNORT-xxx
TC-WAZUH-xxx
TC-ANALYSIS-xxx
TC-TUNE-xxx

4.8 Incident

INC-YYYYMMDD-NNN

예:

INC-20260824-001

4.9 PCAP

PCAP-YYYYMMDD-ATK-NNN.pcap

예:

PCAP-20260824-ATK-003.pcap

5. Host 상세설계

DES-HOST-001 Windows Host

항목	설계값
Role	Hypervisor + Analyst Workstation + Docker Host
OS Target	Windows 11 x64

항목	설계값
Edition	Pro / Enterprise / Education
Version	24H2 이상 권장
Exact Build	[RUNTIME VERIFICATION REQUIRED]
Hyper-V	Enabled
WSL	WSL2
Docker Desktop	4.87.0
Git	Current stable
Wireshark	Current stable
Browser	Edge/Chrome
Hostname	기존 Hostname 유지

Hyper-V는 Windows 11 Pro/Enterprise 환경을 공식 지원하며 Windows 11 Education에도 Hyper-V가 포함된다.

6. Host Storage Layout

Windows Host에서 다음 구조를 사용한다.

```
C:\SOC-Lab\
├── repo\
│   └── soc-detection-monitoring-lab\
├── artifacts\
│   ├── pcaps\
│   ├── logs\
│   ├── backups\
│   └── exports\
├── vm\
│   ├── soc-attacker\
│   ├── soc-gateway\
│   ├── soc-victim\
│   └── soc-sensor\
└── iso\
```

Repository Root

```
C:\SOC-Lab\repo\soc-detection-monitoring-lab
```

Evidence Root

```
C:\SOC-Lab\repo\soc-detection-monitoring-lab\evidence
```

Raw Artifact Root

C:\SOC-Lab\artifacts

Raw PCAP과 대용량 Log는 Git Repository에 직접 저장하지 않는다.

7. VM 상세설계

7.1 VM Allocation

VM	OS	vCPU	RAM	Disk	Generation
soc-gateway	Ubuntu Server 22.04.5	1	1 GB	16 GB	2
soc-attacker	Kali Linux 2026.2	2	4 GB	40 GB	2
soc-victim	Ubuntu Server 22.04.5	2	4 GB	30 GB	2
soc-sensor	Ubuntu Server 22.04.5	4	8 GB	80 GB	2

Kali 2026.2는 2026-06-29 release이다.

Ubuntu 22.04는 Suricata 8.0.6의 Tier 1 Linux 지원 대상이며 Ubuntu의 표준 유지보수는 2027년까지 제공된다.

8. VM Dynamic Memory

모든 Security Lab VM에서

Dynamic Memory = Disabled

로 설정한다.

이유

- Packet Capture 성능 재현성
- Suricata Memory 상태 안정성
- Test 시 Resource 변화 제거
- Evidence 재현성

9. Hyper-V vSwitch 설계

vSwitch	Type	Host Access	Internet	목적
soc-vsw-mgmt	Internal	O	X	관리통신

vSwitch	Type	Host Access	Internet	목적
soc-vsw-attack	Private	X	X	공격망
soc-vsw-victim	Private	X	X	피해자망

10. MGMT Network

ZONE-MGMT
10.77.10.0/24

역할

- Windows Host
- Gateway Management NIC
- Sensor Management NIC
- Wazuh Service

11. Attack Network

ZONE-ATTACK
10.77.20.0/24

역할

Attacker
↔
Gateway

Host 직접 접근 불가.

12. Victim Network

ZONE-VICTIM
10.77.30.0/24

역할

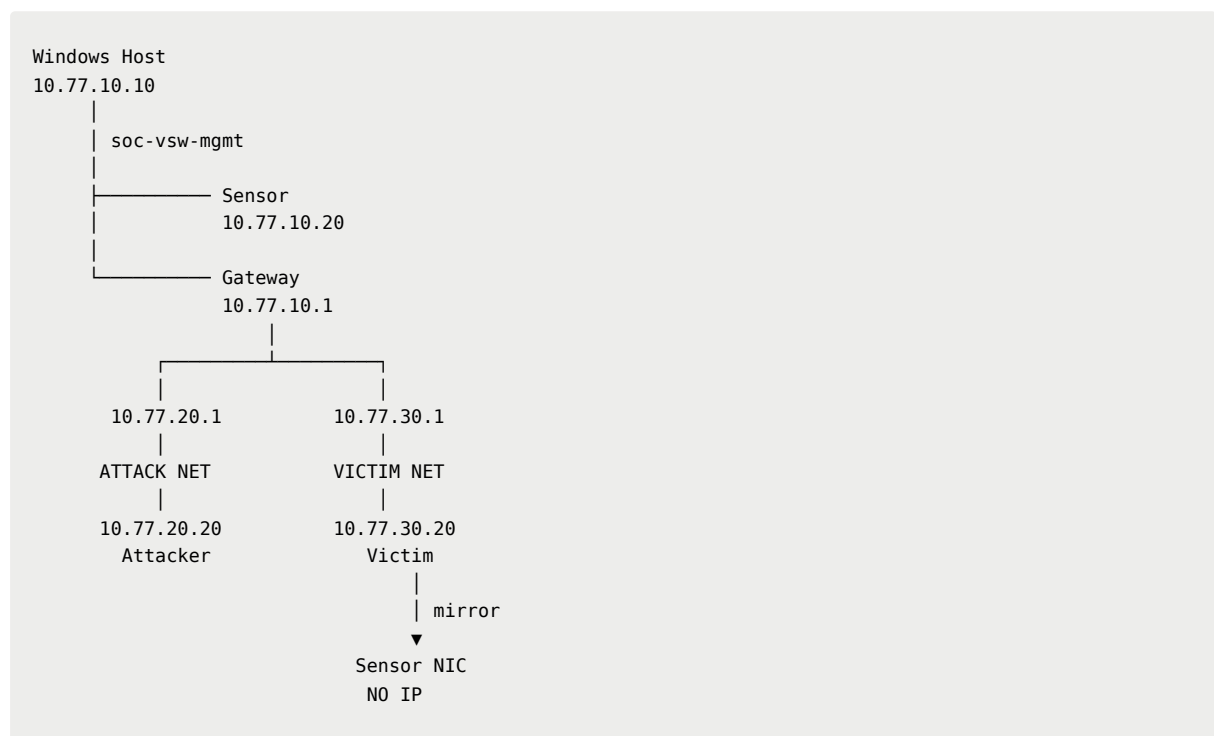
Gateway
↔
Victim

Sensor Monitoring NIC도 동일 vSwitch에 연결하되 IP를 설정하지 않는다.

13. IP Address Plan

Host	NIC	IP	Gateway
Windows Host	vEthernet(soc-vsw-mgmt)	10.77.10.10/24	NONE
Gateway	nic-mgmt	10.77.10.1/24	NONE
Gateway	nic-attack	10.77.20.1/24	NONE
Gateway	nic-victim	10.77.30.1/24	NONE
Attacker	nic-attack	10.77.20.20/24	10.77.20.1
Victim	nic-victim	10.77.30.20/24	10.77.30.1
Sensor	nic-mgmt	10.77.10.20/24	NONE
Sensor	nic-monitor	NO IP	NONE

14. Logical Network Diagram



15. Windows Host Routing

Victim Wazuh Agent가 Windows Host의 Docker Published Port에 접근할 경우 Host는 Victim Network로 Reply Route가 필요하다.

따라서 다음 Logical Static Route를 사용한다.


```
10.77.30.0/24  
via  
10.77.10.1
```

Attack Network에 대한 Host Route는 생성하지 않는다.

```
Windows Host  
X  
10.77.20.0/24
```

16. Gateway Interface 예상값

Ubuntu 설치 후 실제 Linux Interface Name은 Hyper-V Enumeration에 따라 달라질 수 있다.

예상:

```
nic-mgmt → eth0 / ens160 계열  
nic-attack → eth1 / ens192 계열  
nic-victim → eth2 / ens224 계열
```

상태

```
[RUNTIME VERIFICATION REQUIRED]
```

검증:

```
ip -br link  
ip -br addr
```

Hyper-V Adapter Name과 Linux MAC Address를 비교해 Mapping한다.

17. Sensor Interface

논리 명칭:

```
MGMT → sensor-mgmt  
MONITOR → sensor-monitor
```

예상 Linux Name:

```
MGMT → eth0  
MONITOR → eth1
```

실제값

[RUNTIME VERIFICATION REQUIRED]

Implementation Gate:

| [ip link](#) 및 Hyper-V MAC Mapping 후 실제 Monitoring Interface를 Suricata 설정에 반영한다.

18. Gateway Routing

Gateway에서 IPv4 Forwarding을 활성화한다.

논리값:

```
net.ipv4.ip_forward = 1
```

Routing Flow:

```
10.77.20.0/24
  ↓
soc-gateway
  ↓
10.77.30.0/24
```

NAT는 적용하지 않는다.

19. Firewall Engine

Gateway Firewall은

nftables

를 사용한다.

선정 이유

- 현대 Linux Netfilter 관리 방식
- iptables 대비 통합 Ruleset 관리
- Network Security Portfolio 학습가치
- 상태기반 Filtering 구현 가능

20. Gateway Firewall 기본정책

```
INPUT    = DROP
FORWARD  = DROP
```

OUTPUT = ACCEPT

21. Forwarding Policy

Allow

Attacker
10.77.20.20

→

Victim
10.77.30.20

Lab Simulation Traffic 허용.

Return Traffic

ct state established,related
ALLOW

Explicit Deny

10.77.20.0/24

→

10.77.10.0/24

DROP

Attack Zone에서 Management Zone 직접 접근 금지.

22. Victim → Wazuh Policy

Victim에서 다음 목적지에 대해서만 Management Network 접근을 허용한다.

Destination:
10.77.10.10

Allowed:

TCP 1514
TCP 1515

P1 단계에서 Wazuh Agent 사용 시 적용한다.

23. Sensor → Wazuh Policy

Sensor는 같은 MGMT Subnet의 Host로 직접 접근한다.

```
10.77.10.20  
→  
10.77.10.10
```

Allowed:

```
1514/TCP  
1515/TCP
```

24. Hyper-V Port Mirroring

DES-MIRROR-001

Source

```
VM:  
soc-victim  
  
Adapter:  
nic-victim
```

Destination

```
VM:  
soc-sensor  
  
Adapter:  
nic-monitor
```

vSwitch

```
soc-vsw-victim
```

Source Mode

```
PortMirroring = Source
```

Destination Mode

```
PortMirroring = Destination
```

Hyper-V는 Source Adapter의 송수신 Packet을 Destination Adapter로 복제하며 두 Adapter는 같은 vSwitch에 연결되어 있어야 한다.

25. Mirror Validation

Suricata 설치 전에 반드시 Mirror 자체를 검증한다.

```
Attacker
→
Victim

    ↓ Mirror

Sensor tcpdump
```

Test

```
TC-MIRROR-001
```

Expected

Sensor Monitoring NIC에서 다음 Packet 확인:

```
10.77.20.20
→
10.77.30.20
```

Evidence

```
EV-MIRROR-001
```

26. Attacker VM

VM

```
soc-attacker
```

OS

Kali Linux 2026.2

IP

10.77.20.20/24

Gateway

10.77.20.1

기본 Tool

- ping
- curl
- nmap
- netcat
- hping3

제한

Internet = Disabled
MGMT Access = Denied
Victim = Authorized Lab Target

27. Victim VM

soc-victim

OS

Ubuntu Server 22.04.5

IP

10.77.30.20/24

Gateway

10.77.30.1

28. Victim Service

MVP Service:

SSH : 22/TCP
HTTP : 80/TCP

P1:

Wazuh Agent

Test Application은 별도 Container 또는 Python/HTTP test server로 확장할 수 있다.

29. Sensor VM

VM

soc-sensor

OS

Ubuntu Server 22.04.5

vCPU

4

RAM

8 GB

Disk

80 GB

30. Sensor Software

```
Suricata 8.0.6
Snort 3.12.2.0
libDAQ 3.0.27
tcpdump
tshark
jq
Wazuh Agent 4.14.7
```

31. Suricata Installation Source

Ubuntu에서는 OISF의

```
ppa:oisf/suricata-stable
```

을 사용한다.

공식 Suricata 문서는 해당 PPA가 최신 Stable Release를 제공한다고 안내한다.

Version Verification Gate

설치 직후:

```
suricata --build-info
```

결과:

```
8.0.6
```

이어야 한다.

32. Suricata Service

```
Service:
suricata.service
```

Config

```
/etc/suricata/suricata.yaml
```

Main Log Directory

```
/var/log/suricata/
```


33. Suricata Monitoring Interface

[RUNTIME VERIFICATION REQUIRED]

Expected:

eth1

Logical:

sensor-monitor

검증 절차:

1. MAC 확인
2. Hyper-V nic-monitor MAC 비교
3. tcpdump Packet 확인
4. Suricata Interface 지정

34. Suricata Capture

Capture Method:
AF_PACKET

Suricata 8.0.6에서 AF_PACKET은 Tier 1 지원 Capture Method이다.

Flow:

```
Hyper-V Mirror
  ↓
sensor-monitor
  ↓
AF_PACKET
  ↓
Suricata
```

35. Suricata Network Variables

HOME_NET

10.77.30.0/24

EXTERNAL_NET

```
!$HOME_NET
```

이유

Victim Network를 보호 대상 Network로 정의한다.

Attack Network 및 외부 Source는 EXTERNAL_NET으로 취급한다.

36. Suricata Rule Directory

Official/Managed Rules:

```
/var/lib/suricata/rules/suricata.rules
```

Suricata-Update는 기본적으로 해당 위치에 통합 Rule 파일을 관리한다.

37. Custom Rule Directory

```
/etc/suricata/rules/
```

파일:

```
soc-network.rules  
soc-web.rules  
soc-auth.rules  
soc-lab.rules
```

Suricata는 Managed Rule과 별도의 Custom Rule File을 `rule-files` 항목에서 포함할 수 있다.

38. Repository Rule Path

```
suricata/  
└─ rules/  
    ├── soc-network.rules  
    ├── soc-web.rules  
    ├── soc-auth.rules  
    └─ soc-lab.rules
```

39. Suricata SID Policy

Suricata Custom Rule:

9,000,000
~
9,099,999

이 범위는 **본 프로젝트 자체 할당 영역**이며 Suricata 공식 Reserved Range라는 의미가 아니다.

Rule 배포 전 반드시 Duplicate SID 검사를 수행한다.

Category:

범위	Category
9000000-9009999	Network
9010000-9019999	Web
9020000-9029999	Authentication
9030000-9039999	Lab
9090000-9099999	Reserved

40. Suricata EVE

Primary Output:

```
/var/log/suricata/eve.json
```

EVE는 JSON 기반 Event Output이며 `event_type`, Source/Destination, Protocol 및 Alert Metadata를 기록한다.

41. EVE Enabled Events

Event	Status
alert	Enabled
flow	Enabled
http	Enabled
dns	Enabled
tls	Enabled
stats	Enabled

42. Other Suricata Logs

```
/var/log/suricata/fast.log
```

사용 목적:

- 즉시 Alert 확인
- Troubleshooting

Primary Source of Truth는

```
eve.json
```

으로 유지한다.

43. PCAP Runtime Storage

Sensor:

```
/var/lib/soc-lab/pcaps/
```

구조:

```
/var/lib/soc-lab/pcaps/
├─ raw/
├─ scenarios/
├─ validated/
└─ archive/
```

44. PCAP Repository

Git에는 PCAP Metadata와 소형 Sanitized Sample만 저장한다.

```
pcaps/
├─ samples/
└─ metadata/
```

대용량 Raw PCAP은 제외한다.

45. Suricata Log Rotation

EVE

```
Rotation:
Daily OR 500 MB

Retention:
30 files
```

Compression:
Enabled

fast.log

Daily
14 files
compress

PCAP

Raw:
14 days

Validated Scenario:
Project Lifetime

46. Suricata Config Validation

Test:

TC-SURI-CONFIG-001

검증 명령 유형:

```
suricata -T -c /etc/suricata/suricata.yaml
```

Expected:

Configuration valid
No fatal rule error

47. Suricata Capture Test

TC-SURI-001

Procedure

Attacker
→ ping/curl

→ Victim

Expected

Suricata stats packet count 증가

Evidence:

EV-SURI-001

48. EVE Test

TC-SURI-002

Expected:

/var/log/suricata/eve.json

에 최신 Timestamp Event 생성.

49. Custom Rule Test

TC-SURI-003

Flow:

Custom Traffic
→ Rule Match
→ EVE Alert

Evidence:

EV-SURI-003

50. Snort 상세설계

Version

3.12.2.0

libDAQ

3.0.27

Snort 공식 Download 페이지 기준이다.

51. Snort Runtime

Snort는 상시 Service가 아니라 **On-demand Offline Validation Engine**으로 운용한다.

PCAP
→
Snort
→
Alert JSON
→
Suricata 비교

52. Snort Path

Binary:

/usr/local/bin/snort

Config:

/usr/local/etc/snort/snort.lua

Rule:

/usr/local/etc/snort/rules/

Local:

/usr/local/etc/snort/rules/soc-local.rules

53. Snort SID

Snort는 공식적으로 0-999999 를 배포 Rule용으로 예약하고 Local Rule은 1000000 이상을 사용하도록 권고한다.

본 프로젝트에서는 충돌 방지를 위해 다음 범위를 배정한다.

```
9,100,000
~
9,199,999
```

54. Snort PCAP Input

```
/var/lib/soc-lab/pcaps/validated/
```

55. Snort Output

```
/var/log/snort/
```

기본 Portfolio Output:

```
alert_json
```

Snort 3는 alert_json Logger를 공식 지원한다.

56. Snort Configuration Validation

```
TC-SNORT-CONFIG-001
```

Command Pattern:

```
snort -c /usr/local/etc/snort/snort.lua -T
```

Snort 공식 Command Line 문서는 -T 를 Configuration Validation 옵션으로 정의한다.

57. Snort PCAP Validation

```
snort
-c /usr/local/etc/snort/snort.lua
-r <pcap>
```



```
-R <rule>
-A alert_json
```

Snort 공식 문서는 `-r` 로 PCAP Readback을 지원한다.

58. IDS Comparison Output

Repository:

```
docs/
├── 06-detection/
│   └── comparisons/
```

Comparison Template:

항목	Suricata	Snort
Scenario		
Detection		
SID		
Message		
Alert Count		
Source		
Destination		
Protocol		
FP 가능성		
Analyst Note		

59. Docker Desktop

Version:

```
4.87.0
```

Backend:

```
WSL2
```

Docker Desktop 공식 문서에서 WSL 2.1.5 이상이 최소 요구사항이다.

60. Wazuh Architecture

```
Docker Desktop
  ↓
soc-wazuh-net
  ├── soc-wazuh-manager
  ├── soc-wazuh-indexer
  └── soc-wazuh-dashboard
```

61. Wazuh Version

4.14.7

Images:

```
wazuh/wazuh-manager:4.14.7
wazuh/wazuh-indexer:4.14.7
wazuh/wazuh-dashboard:4.14.7
```

공식 v4.14.7 Compose가 해당 image를 사용한다.

62. Wazuh Runtime Root

WSL2 내부:

```
/home/soclab/wazuh-docker/
```

Vendor Source:

```
/home/soclab/wazuh-docker/v4.14.7/
```

원칙

Vendor Compose는 직접 대규모 수정하지 않는다.

Custom 변경:

```
docker-compose.override.yml
```

로 분리한다.

63. Wazuh Compose Project

```
COMPOSE_PROJECT_NAME=soc-wazuh
```

Docker Network:

```
soc-wazuh-net
```

64. Wazuh Manager Container

```
soc-wazuh-manager
```

Image:

```
wazuh/wazuh-manager:4.14.7
```

65. Wazuh Indexer Container

```
soc-wazuh-indexer
```

Image:

```
wazuh/wazuh-indexer:4.14.7
```

66. Wazuh Dashboard Container

```
soc-wazuh-dashboard
```

Image:

```
wazuh/wazuh-dashboard:4.14.7
```

67. Wazuh Port Mapping

Official Wazuh Docker ports는 1514, 1515, 514/UDP, 55000, 9200, 443이다.

본 Lab에서는 최소 노출 원칙을 적용한다.

Host Bind	Container	상태
10.77.10.10:1514/TCP	Manager 1514	ENABLE
10.77.10.10:1515/TCP	Manager 1515	ENABLE
10.77.10.10:443/TCP	Dashboard 5601	ENABLE
127.0.0.1:55000/TCP	Manager 55000	LOCAL ONLY
127.0.0.1:9200/TCP	Indexer 9200	LOCAL ONLY
UDP 514	Manager 514	DISABLED initially

Note

IP-specific Docker Desktop Port Binding은 Implementation 단계에서

[RUNTIME VERIFICATION REQUIRED]

한다.

실패 시 Windows Firewall을 이용해 동일 Security Boundary를 유지한다.

68. Wazuh Agent Ports

Agent:

1514/TCP

Enrollment:

1515/TCP

Wazuh 공식 Agent Enrollment Requirements와 Agent Connection 문서의 기본값이다.

69. Wazuh Volumes

공식 v4.14.7 Compose의 Persistence 구조를 유지한다.

Named Volumes:

```
soc-wazuh-api-configuration
soc-wazuh-etc
soc-wazuh-logs
soc-wazuh-queue
soc-wazuh-var-multigroups
soc-wazuh-integrations
soc-wazuh-active-response
soc-wazuh-agentless
soc-wazuh-wodles
soc-wazuh-filebeat-etc
soc-wazuh-filebeat-var
```

```
soc-wazuh-indexer-data
soc-wazuh-dashboard-config
soc-wazuh-dashboard-custom
```

70. Wazuh Indexer Data

Container Path:

```
/var/lib/wazuh-indexer
```

Source:

```
soc-wazuh-indexer-data
```

71. Wazuh Manager Data

주요 Runtime:

```
/var/ossec/etc
/var/ossec/logs
/var/ossec/queue
/var/ossec/integrations
/var/ossec/active-response/bin
```

해당 Mount 구조는 공식 4.14.7 Compose 기준이다.

72. Wazuh Secret Policy

Repository:

```
.env.example
```

Runtime:

```
.env
```

.env :

```
GIT EXCLUDE
```

73. .gitignore Baseline

```
.env
*.key
*.pem
*.p12
*.pfx
secrets/
credentials/
runtime/
logs/
pcaps/raw/
pcaps/archive/
*.pcap
*.pcapng
```

단 Sanitized Sample PCAP은 명시적으로 Allow할 수 있다.

74. Suricata → Wazuh

Sensor Wazuh Agent:

```
/var/ossec/etc/ossec.conf
```

Log Source:

```
/var/log/suricata/eve.json
```

Logical configuration:

```
<localfile>
  <log_format>json</log_format>
  <location>/var/log/suricata/eve.json</location>
</localfile>
```

이는 Wazuh 공식 Suricata Integration 방식과 동일하다.

75. Wazuh Sensor Agent

Agent Name:

```
soc-sensor
```

Manager:

10.77.10.10

Communication:

1514/TCP

Enrollment:

1515/TCP

76. Victim Wazuh Agent

Agent Name:

soc-victim

Manager:

10.77.10.10

Traffic:

```
Victim
10.77.30.20
  ↓
Gateway
  ↓
10.77.10.10
```

Firewall:

1514/TCP ALLOW
1515/TCP ALLOW

77. Wazuh Dashboard

URL:

<https://10.77.10.10/>

Access:

Windows Analyst Host only

Attack/Victim Network 직접 Access 금지.

78. Wazuh Indexer

Host External Access:

127.0.0.1 only

Container Internal:

<https://wazuh.indexer:9200>

79. Wazuh API

External:

127.0.0.1:55000

Remote Attack/Victim Access:

DENY

Wazuh API는 JWT 인증을 요구한다.

80. Wazuh Resource Allocation

Official Docker Minimum:

4 CPU
8 GB RAM
50 GB Disk

LLD Allocation

Docker CPU:
4

Docker Memory:
10 GB Target


```
Indexer Storage:  
60 GB Initial
```

Host RAM은 **32GB** 권장이다.

81. WSL Requirement

```
WSL >= 2.1.5
```

Implementation 전에:

```
wsl --version
```

검증.

Docker는 가능하면 최신 WSL 사용을 권장한다.

82. Repository Structure

```
soc-detection-monitoring-lab/  
├── README.md  
├── .gitignore  
├── .env.example  
├── docs/  
│   ├── 01-requirements/  
│   ├── 02-architecture/  
│   ├── 03-design/  
│   ├── 04-deployment/  
│   ├── 05-testing/  
│   ├── 06-detection/  
│   │   └── comparisons/  
│   ├── 07-investigation/  
│   ├── 08-incident/  
│   └── 09-troubleshooting/  
├── infrastructure/  
│   ├── hyper-v/  
│   ├── network/  
│   └── docker/  
├── suricata/  
│   ├── config/  
│   ├── rules/  
│   └── scripts/  
├── snort/  
│   ├── config/  
│   ├── rules/  
│   └── scripts/  
├── wazuh/  
│   └── config/
```

```
|   └─ docker/
|
|   └─ attack-lab/
|       ├── scenarios/
|       └── scripts/
|
|   └─ pcaps/
|       ├── samples/
|       └── metadata/
|
|   └─ tests/
|
|   └─ evidence/
|
|   └─ scripts/
```

83. Runtime / Repository 분리

Repository

```
C:\SOC-Lab\repo\
```

Runtime

Linux:

```
/etc/suricata
/var/log/suricata
/usr/local/etc/snort
/var/log/snort
/var/lib/soc-lab
```

Docker:

```
/home/soclab/wazuh-docker
```

84. Detection Rule Metadata

```
rule_id:
engine:
sid:
rev:
name:
category:
severity:
protocol:
```

```

source:
destination:
condition:
mitre_tactic:
mitre_technique:
validation_scenario:
validation_pcap:
false_positive:
created:
modified:
status:
git_commit:

```

85. MITRE ATT&CK Metadata

```

Tactic
Technique ID
Technique Name
Sub-technique
Mapped Rule
Mapped Alert
Mapped Incident
Evidence

```

ATT&CK v19.2를 기준으로 한다.

86. MVP Scenario

LLD Baseline Scenario:

Scenario	방법	Purpose
ATK-001 ICMP	Live	Connectivity
ATK-003 Port Scan	Live	Recon Detection
ATK-005 Auth Failure	Live	Endpoint + IDS
ATK-007 Suspicious HTTP	Synthetic Live	Web Detection
ATK-012 Malware Traffic	Offline PCAP	Threat Analysis

87. Test Case Template

```

Test ID:
Objective:
Requirement:
Design:
Precondition:
Input:

```

Procedure:
Expected:
Actual:
Result:
Evidence:

88. Core Test Matrix

Test	검증
TC-HOST-001	Hyper-V 활성화
TC-NET-001	Attacker → Gateway
TC-NET-002	Attacker → Victim
TC-NET-003	Attack → MGMT 차단
TC-MIRROR-001	Victim Traffic Mirror
TC-SURI-001	Packet Capture
TC-SURI-002	EVE 생성
TC-SURI-003	Custom Rule Alert
TC-PCAP-001	PCAP 생성
TC-SNORT-001	Snort Offline Detection
TC-WAZUH-001	Agent Enrollment
TC-WAZUH-002	EVE Ingestion
TC-WAZUH-003	Dashboard Search
TC-ANALYSIS-001	Alert → PCAP 연결
TC-TUNE-001	FP Tuning

89. Health Check

Hyper-V

VM State
vSwitch 존재
NIC 연결

Network

ip addr
ip route
ping

Mirror

```
tcpdump
```

Monitoring NIC Packet Count 증가.

Suricata

```
systemctl status suricata  
suricata -T  
eve.json Timestamp
```

Snort

```
snort -V  
snort -c ... -T
```

Wazuh

```
docker compose ps
```

추가:

```
Agent Status  
Manager  
Indexer  
Dashboard
```

90. Evidence Structure

예:

```
evidence/  
├── EV-SURI-003/  
│   ├── metadata.md  
│   ├── screenshot.png  
│   ├── event.json  
│   ├── rule.rules  
│   └── pcap-reference.md
```

91. Evidence Metadata

```
Evidence ID:  
Requirement:  
HLD:  
LLD:  
Test:  
Scenario:  
Timestamp:  
Component:  
Description:  
Expected:  
Actual:  
Result:  
PCAP:  
Rule:  
Commit:
```

92. PCAP Metadata

```
PCAP ID:  
Scenario:  
Capture Start:  
Capture End:  
Sensor:  
Source:  
Destination:  
SHA256:  
Related Alert:  
Related Incident:
```

PCAP에는 SHA-256 Hash를 생성해 Evidence 무결성을 확인할 수 있도록 한다.

93. Backup

Backup 대상:

```
Repository  
Custom Rules  
Wazuh Config  
Certificates  
Evidence  
Incident Reports  
Validated PCAP
```

제외:

```
전체 Raw EVE  
전체 Wazuh Index  
모든 Raw PCAP
```

94. Backup Path

Windows:

```
C:\SOC-Lab\artifacts\backups\
```

Naming:

```
BACKUP-YYYYMMDD-TYPE
```

95. Git Branch Policy

```
main
feature/network
feature/suricata
feature/snort
feature/wazuh
feature/detection
feature/evidence
```

96. Commit Convention

```
feat(network):
feat(suricata):
feat(snort):
feat(wazuh):
feat(detection):
test:
docs:
fix:
chore:
```

97. Troubleshooting Tree — Alert 없음

```
Attack Traffic 존재?
↓
Victim Packet 수신?
↓
Mirror 정상?
↓
Sensor NIC 수신?
↓
Suricata Process 정상?
↓
Correct Interface?
↓
Rule Loaded?
```

↓
Rule Condition Match?
↓
EVE Output?

98. Troubleshooting — Wazuh Event 없음

eve.json 생성?
↓
Wazuh Agent 정상?
↓
ossec.conf localfile?
↓
Agent 연결?
↓
1514 reachable?
↓
Manager 수신?
↓
Indexer 정상?
↓
Dashboard Query?

99. Start Order

1. Hyper-V
2. soc-vsw-mgmt
3. soc-vsw-attack
4. soc-vsw-victim
5. soc-gateway
6. soc-victim
7. soc-sensor
8. soc-attacker
9. Network Validation
10. Mirror Validation
11. Suricata
12. Docker Desktop
13. Wazuh Indexer
14. Wazuh Manager
15. Wazuh Dashboard
16. Sensor Wazuh Agent
17. Victim Wazuh Agent

100. Shutdown Order

1. Attack Simulation Stop
2. Victim Wazuh Agent
3. Sensor Wazuh Agent
4. Wazuh Dashboard
5. Wazuh Manager
6. Wazuh Indexer

7. Docker Desktop
8. Suricata
9. Attacker
10. Sensor
11. Victim
12. Gateway

Indexer는 강제 종료하지 않고 정상 종료를 원칙으로 한다.

101. Security Controls

Network

- Attack Network Internet 차단
- Attack → MGMT Deny
- Host → Attack Route 없음
- Monitor NIC IP 없음

Authentication

- Default Wazuh Password 변경
- Enrollment Credential 별도 관리

Secrets

- `.env` Git 제외
- Private Key Git 제외

Logs

- Raw Log 수정 금지
- Evidence에는 원본 Hash 포함

Repository

- Public Upload 전 IP/Credential 검토

102. Port/Protocol Final Matrix

ID	Source	Destination	Port	Purpose
NET-001	Attacker	Victim	Scenario	Lab Traffic
WAZ-001	Sensor	Host/Wazuh	1514/TCP	Agent Events
WAZ-002	Sensor	Host/Wazuh	1515/TCP	Enrollment
WAZ-003	Victim	Host/Wazuh	1514/TCP	Agent Events

ID	Source	Destination	Port	Purpose
WAZ-004	Victim	Host/Wazuh	1515/TCP	Enrollment
WAZ-005	Analyst	Dashboard	443/TCP	SIEM
WAZ-006	Host Local	Manager API	55000/TCP	API
WAZ-007	Host Local	Indexer	9200/TCP	Index API

103. Detailed Design Baseline v1.0

영역	최종 결정
Host OS	Windows 11 x64 Target
Actual Build	[RUNTIME VERIFICATION REQUIRED]
Attacker	soc-attacker / Kali 2026.2
Gateway	soc-gateway / Ubuntu 22.04.5
Victim	soc-victim / Ubuntu 22.04.5
Sensor	soc-sensor / Ubuntu 22.04.5
MGMT vSwitch	soc-vsw-mgmt / Internal
ATTACK vSwitch	soc-vsw-attack / Private
VICTIM vSwitch	soc-vsw-victim / Private
MGMT CIDR	10.77.10.0/24
ATTACK CIDR	10.77.20.0/24
VICTIM CIDR	10.77.30.0/24
Host MGMT IP	10.77.10.10
Gateway MGMT	10.77.10.1
Gateway ATTACK	10.77.20.1
Gateway VICTIM	10.77.30.1
Attacker IP	10.77.20.20
Victim IP	10.77.30.20
Sensor MGMT	10.77.10.20
Sensor Monitor	NONE
Mirror Source	soc-victim / nic-victim
Mirror Destination	soc-sensor / nic-monitor
Capture	Hyper-V Mirror + AF_PACKET
Suricata	8.0.6
Sensor Interface	[RUNTIME VERIFICATION REQUIRED]
HOME_NET	10.77.30.0/24
EXTERNAL_NET	!\$HOME_NET
Suricata Config	/etc/suricata/suricata.yaml
EVE	/var/log/suricata/eve.json

영역	최종 결정
Rule Runtime	/etc/suricata/rules/
Managed Rule	/var/lib/suricata/rules/suricata.rules
Suricata SID	9000000–9099999
Snort	3.12.2.0
Snort Config	/usr/local/etc/snort/snort.lua
Snort SID	9100000–9199999
PCAP Runtime	/var/lib/soc-lab/pcaps
Wazuh	4.14.7
Docker Desktop	4.87.0
Docker Network	soc-wazuh-net
Manager	soc-wazuh-manager
Indexer	soc-wazuh-indexer
Dashboard	soc-wazuh-dashboard
Wazuh Data	1514/TCP
Wazuh Enrollment	1515/TCP
Dashboard	443/TCP
Manager API	localhost:55000
Indexer API	localhost:9200
Repository	C:\SOC-Lab\repo\soc-detection-monitoring-lab
Evidence	Repository\evidence
Raw Artifacts	C:\SOC-Lab\artifacts
ATT&CK	19.2

104. Runtime Verification Required

다음 항목만 실제 구축 시 확인 후 Baseline에 추가한다.

```

HOST Windows Build
WSL Exact Version
Gateway Linux Interface Names
Victim Linux Interface Name
Sensor MGMT Interface
Sensor Monitor Interface
Hyper-V generated MAC addresses
Docker IP-specific bind behavior

```

이 항목은 설계 미완료가 아니라 **Runtime Discovery 영역**으로 정의한다.

105. Implementation Readiness Checklist

P0

- ☒ VM 이름
- ☒ OS
- ☒ CPU/RAM/Disk
- ☒ vSwitch
- ☒ Network Type
- ☒ CIDR
- ☒ IP
- ☒ Gateway
- ☒ Routing
- ☒ Firewall Direction
- ☒ Mirror Source
- ☒ Mirror Destination
- ☒ Sensor NIC Role
- ☒ Suricata Version
- ☒ AF_PACKET
- ☒ HOME_NET
- ☒ Rule Path
- ☒ SID Policy
- ☒ EVE Path
- ☒ PCAP Path
- ☒ Snort Version
- ☒ Snort Rule Policy
- ☒ Evidence Structure
- ☒ Test ID
- ☒ Repository Structure

P1

- ☒ Wazuh Version
- ☒ Container Architecture
- ☒ Docker Network
- ☒ Port Policy
- ☒ Agent Pipeline

- ☒ Dashboard
- ☒ Volume Strategy
- ☒ Credential Policy
- ☒ ATT&CK Version

106. Final Readiness Decision

LLD DESIGN STATUS:
COMPLETE

단 다음 Runtime Gate는 실제 Implementation Phase에서 닫는다.

RG-001 Host OS Build
RG-002 Linux NIC Mapping
RG-003 Hyper-V Port Mirror
RG-004 Docker Host Bind

따라서 현재 상태는:

**IMPLEMENTATION READY WITH RUNTIME VERIFICATION
GATES**

로 선언한다.

107. 다음 단계

다음 산출물은

**「보안관제 프로젝트 구축계획서 / Implementation Plan
v1.0」**

이다.

LLD에서 기술결정은 종료되었다.

이후부터는 실제 작업을 다음 Phase로 분해한다.

Phase 0
Host Readiness

Phase 1
Repository Baseline

Phase 2
Hyper-V vSwitch

Phase 3

VM Provisioning

Phase 4

IP / Routing / nftables

Phase 5

Port Mirroring

Phase 6

Packet Visibility Gate

Phase 7

Suricata Deployment

Phase 8

Suricata Detection Validation

Phase 9

PCAP / Snort Environment

Phase 10

Wazuh Docker Deployment

Phase 11

Sensor Wazuh Agent

Phase 12

Victim Wazuh Agent

Phase 13

Suricata → Wazuh Integration

Phase 14

Attack Scenarios

Phase 15

SOC Investigation

Phase 16

False Positive Tuning

Phase 17

Evidence / Validation

Phase 18

Portfolio Completion

각 Phase는 다음 형식으로 작성한다.

Objective

Prerequisite

Input

Tasks

Commands

Expected Result

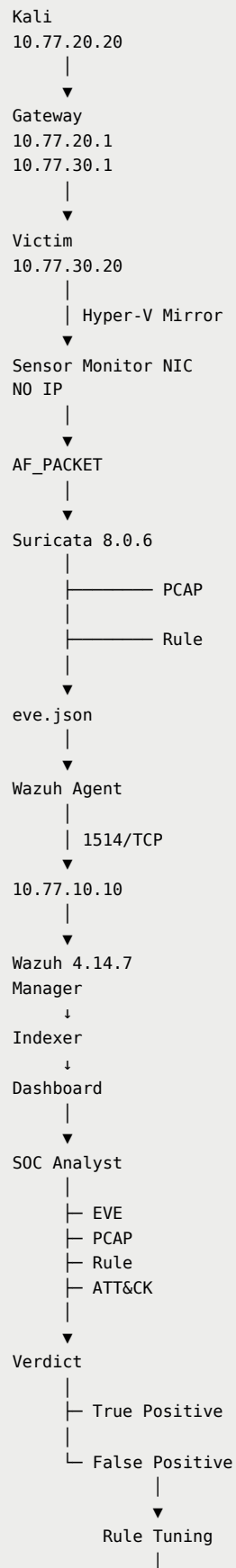
Test Case

Evidence

Rollback

Completion Gate

108. 최종 LLD 핵심 구현 구조



▼
Re-test
|
▼
Evidence

LLD Baseline v1.0 — FINAL

본 프로젝트는 Hyper-V의 `soc-vsw-attack`, `soc-vsw-victim`, `soc-vsw-mgmt` 세 Network로 공격·피해·관리 영역을 분리한다. `soc-attacker(10.77.20.20)` 에서 발생한 Traffic은 `soc-gateway` 를 거쳐 `soc-victim(10.77.30.20)` 으로 전달되고, Victim vNIC를 Hyper-V Port Mirroring Source로 설정하여 `soc-sensor` 의 IP 없는 Monitoring NIC로 Packet을 복제한다. Suricata 8.0.6은 해당 Interface를 AF_PACKET으로 수집하고 HOME_NET `10.77.30.0/24` 를 기준으로 Detection을 수행한다. 탐지 Event의 원본은 `/var/log/suricata/eve.json`, Network 원본 Evidence는 PCAP으로 보존한다. EVE는 Sensor의 Wazuh Agent 4.14.7을 통해 Windows Host의 Docker 기반 Wazuh 4.14.7 Single-node로 전달된다. Snort 3.12.2.0은 동일 PCAP에 대한 Secondary Validation Engine으로 운영한다. 모든 Rule, Test, PCAP Metadata, Alert 분석 및 Detection Tuning 결과는 Requirement → Design → Test → Evidence 관계로 Git Repository에 기록한다.

Detailed Design Status

IMPLEMENTATION READY WITH RUNTIME VERIFICATION GATES